

Incremental EWC learning on a microcontroller: measured FP32 parity, collapse of naive quantization and recovery via a calibrated INT8 kernel

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

September 9, 2026

Abstract

We study the porting of an *Elastic Weight Consolidation* (EWC) classifier for predictive maintenance onto a Cortex-M4 microcontroller (NUCLEO-F439ZI, 256 kB SRAM, no NPU). Three results are established under a **paired** protocol (same seed, same sample order, same normalization) between the PC run and the board. **(1)** In FP32, the PC $\leftrightarrow$ board parity is *exact* in the frozen protocol (1.000) and near-exact online (0.963–0.989), with an inference + online update latency $\ll$ 100 ms (Gap 2). **(2)** *Naive* post-training quantization (PTQ) of the binary EWC head collapses on the board: the F1 score drops from $\approx$ 0.92 to 0.138 and 0.1337. **(3)** A bit-exact PC-emulated ablation isolates the cause (int8 accumulator, uncalibrated per-tensor scale) and demonstrates *recovery* through a calibrated kernel: the `per_tensor_calib` scheme restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201), and the `q15` scheme matches FP32. **(4)** That recovery is now *measured on-board* over four per-channel cells (F1 0.9173 Monitoring, 0.8995 Pronostia; frozen parity 1.000 against the emulator; no CRC error). We finally extend the study to the quantization *moment*, to sub-INT8 *depth* and to the full system budget: weight RAM is divided by 4, yet total RAM is unchanged; latency *increases* (74 μ s against 48–50 μ s, requantization weighing a third of the budget) and energy does not move (67.47 μ J against 67.65 μ J). We systematically distinguish *on-board* measurements from *bit-exact PC emulation*, and explicitly leave "to be measured" the cells not yet streamed.

Keywords: continual learning; EWC; INT8 quantization; PTQ; microcontroller; predictive maintenance; TinyML.

1 Introduction

Industrial predictive maintenance increasingly requires models able to *learn continually* on the equipment itself, without a round-trip to a server: operating conditions drift (wear, regime or machine change), and a frozen model degrades. Continual Learning (CL) answers this need [9, 3], yet its deployment on an Microcontroller Unit (MCU) remains constrained: Static Random Access Memory (SRAM) in the range of a hundred kilobytes, no neural accelerator, a strict latency budget.

Against these constraints, INT8 quantization is the best-established reflex of the embedded literature [13, 2]. It is almost always justified by a threefold argument — less memory, less time, less energy — that is seldom verified *on the target* for each of the three terms separately. This is precisely what we measure here, and the three terms do not behave alike.

This work targets a **triple gap** that few studies close simultaneously: (Gap 1) validation on real industrial time series; (Gap 2) operation under a *strict and measured* memory and latency budget — here 256 kB of SRAM and 100 ms per inference + update cycle; (Gap 3) INT8 quantization compatible with incremental training. We instantiate these three axes on an Elastic Weight Consolidation (EWC) classifier [5] ported to C on a NUCLEO-F439ZI board (Cortex-M4 180 MHz, 256 kB SRAM, no Neural Processing Unit (NPU)), evaluated on two industrial datasets with contrasting continual scenarios.

Contributions. (i) A *paired* PC↔board protocol (same seed, same sample order, same normalization) establishing an *exact* FP32 parity in the frozen protocol (1.000), near-exact online (from 0.9626 to 0.9887 across the three cells), for an inference + learning latency $\ll$ 100 ms. (ii) The demonstration, *on real hardware*, of the collapse of naive Post-Training Quantization (PTQ) of the binary EWC head: F1 falls from ≈ 0.92 to 0.138 and 0.1337 even though weight Random Access Memory (RAM) is indeed divided by 4. (iii) The isolation of the cause through a *bit-exact emulated* ablation, then the *recovery measured on-board* with a calibrated-scale kernel: F1 returns to 0.9173 and 0.8995, with a frozen parity of 1.000 against the emulator and no Cyclic Redundancy Check (CRC) error. (iv) Two complementary quantization axes: the *moment*, where Quantization-Aware Training (QAT) training adds no decisive gain over a properly calibrated PTQ, and the *depth*, where the boundary depends on the metric chosen — ternary in AUROC, INT4 in on-board measured F1 — and where the memory gain exists only with genuine bit-packing. (v) A complete system budget measured on the same board, which *decouples the three promises of INT8*: weight RAM is indeed divided by 4, but total system RAM is unchanged, latency increases by about half, and energy does not move.

This last point is the guiding thread of the article. On this target, **INT8**

is justified by memory, neither by speed nor by energy — and even then, by a memory whose nature must be stated. Throughout the text and the captions we systematically distinguish what is *measured on-board* from what is *bit-exactly emulated on PC*, and we explicitly leave "to be measured" any cell not yet streamed, rather than estimating it.

Organization. Section 2 positions the work, Section 3 describes the model, the paired protocol and the quantization schemes, Section 4 the setup. Results come in three stages: parity, collapse and recovery (Section 5), moment and depth (Section 6), system budget (Section 7). Section 8 discusses both paradoxes and the limitations.

2 Related work

Embedded continual learning. CL methods are classically split into regularization-, replay- and architecture-based approaches [9]. EWC [5] belongs to the first family: a quadratic penalty weighted by the Fisher information protects the parameters that matter for past tasks, without storing any data — a valuable property under memory constraints. On MCUs, Tiny Online Learning (TinyOL) [14] demonstrated the feasibility of online learning of an additional layer, and Hyperdimensional Computing (HDC) [1] that of very low-power non-neural learning. LifeLearner [7] and quantized latent-replay platforms [13] address the memory/accuracy trade-off on embedded targets. Recent Tiny Machine Learning (TinyML) surveys [2] point out that *on-device* training remains under-explored compared with inference alone, and work applied to predictive maintenance [3] is still mostly evaluated off-target.

Quantizing: three degrees of freedom, rarely crossed. Quantization is often reduced to a choice of *format* — FP32 to INT8, or Q15 fixed point. Two further degrees govern it just as much. The *moment* first [4, 6]: quantizing after training, in PTQ, is immediate but sensitive to scale calibration, whereas quantizing during training, in QAT, costs a retraining and assumes access to the learning pipeline, which is not always available in an industrial context. The *depth* next, together with the granularity that comes with it: going below 8 bits pays off only if the scale is estimated finely enough, per channel rather than per tensor, and if storage actually follows the advertised format. These three axes are most often studied separately; we vary them on *one head, one dataset and one board*, which makes them comparable rather than merely juxtaposed.

What is rarely verified: the three promises, separately. Quantization is usually justified by a threefold argument — less memory, less time,

less energy — inherited from targets equipped with dedicated integer units. On a general-purpose core with an Floating-Point Unit (FPU), nothing guarantees that the three terms behave alike, and reporting them jointly requires instrumentation that few studies put in place: measuring the *total* system memory rather than the weight footprint alone, timing the kernel item by item, and integrating a real current at the probe. Our empirical contribution is to treat these three promises as three distinct questions, measured on the same board: we show that the first holds but bears on a minor item, and that the other two do not. Along the way we establish the collapse of naive PTQ of a decision head, isolate its cause rather than masking it with upstream QAT, and demonstrate its recovery through a calibrated kernel.

3 Method

3.1 EWC formulation and embedded adaptation

Elastic penalty. EWC fights catastrophic forgetting by anchoring the parameters deemed important for what has already been learned [5]. After a task, we keep an anchor point $\theta^\star$ — the parameters as they stood at the end of that task — and an estimate F of the importance of each coordinate. Training on the next task then minimizes the task loss augmented with a quadratic pull towards that anchor:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{task}}(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^\star)^2. \quad (1)$$

The coefficient λ arbitrates between plasticity and retention. Each coordinate is pulled back with its own stiffness, λF_i : a coordinate of high importance becomes expensive to move, a coordinate of low importance stays free. The sum runs over *all* parameters of the head, weights and biases alike. There is a single quadratic term, anchored on the latest $\theta^\star$, rather than a sum over past tasks: the memory cost of the regularizer is therefore constant, two copies of the parameters, whatever the number of tasks already seen. On the first task, F and $\theta^\star$ do not exist yet and the penalty is absent.

Estimating importance on PC. F is the diagonal of the Fisher information matrix, estimated at the end of a task on that task’s data. We use its empirical form, that is, evaluated on the real labels rather than on labels drawn from the model. The quantity actually accumulated is the squared gradient of the loss *averaged over a batch*, averaged over the B batches traversed:

$$F_i = \frac{1}{B} \sum_{b=1}^B \left(\frac{\partial \mathcal{L}_b}{\partial \theta_i} \right)^2, \quad \mathcal{L}_b = \frac{1}{|b|} \sum_{n \in b} \ell(x_n, y_n; \theta). \quad (2)$$

The budget is capped at 200 samples, which is enough to rank the coordinates against one another — only that ranking matters, since λ absorbs the global scale. Two writings of the accumulation coexist in the project. The *online* variant in the sense of [15] maintains a single importance and lets it decay, $F \leftarrow \gamma F + F^{(t)}$ with $\gamma = 0.9$, which gradually fades the oldest tasks. The head actually ported to the board instead *replaces* its importance at every consolidation: the anchor always follows the last consolidated task.

What the board can do, and what it cannot. The port keeps the penalty of (1), whose derivative $\lambda F_i(\theta_i - \theta_i^*)$ adds to the task gradient. The embedded update reads, for a weight W of layer l :

$$W_{ji} \leftarrow W_{ji} - \eta \left(\underbrace{\delta_j a_i}_{\text{task gradient}} + \underbrace{\lambda F_{ji}(W_{ji} - W_{ji}^*)}_{\text{elastic pull}} \right), \quad \eta = 0.01. \quad (3)$$

Consolidation, on the other hand, cannot reproduce (2): estimating it would require keeping a batch of labelled samples and running a backward pass over them, neither of which a board in continuous streaming, with no data memory, has at hand. Importance is therefore approximated by the *magnitude* of the weight itself, maintained as an exponential moving average:

$$F_{ji} \leftarrow \alpha F_{ji} + (1 - \alpha) W_{ji}^2, \quad W_{ji}^* \leftarrow W_{ji}, \quad \alpha = 0.99. \quad (4)$$

This substitution must be named for what it is: W^2 is not the Fisher, it is an importance proxy that requires no data, no label and no backward pass, and whose cost is one read-write per weight. It assumes that large-magnitude coordinates are the ones carrying the learned function. Two consequences follow. The embedded penalty covers the weight matrices only, biases having neither importance nor anchor stored, which saves a third of the regularization state. And at initialization $F \equiv 0$: until a consolidation has occurred, the elastic pull is zero and the board learns by plain gradient descent.

One last, purely numerical divergence is worth flagging, as it sheds light on the parity measurements: the board updates each weight in place and then propagates the error to the previous layer using the *already modified* weight, where the PC framework uses the pre-step weights throughout. Under frozen inference, both platforms execute the same function and parity is exact; under the update regime, this divergence and the unit batch are enough to explain why it no longer quite is.

3.2 EWC model and C port

The model is a multilayer perceptron with an EWC head (`EWCmlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) trained incrementally as described in Section 3.1. The head is ported to C (`ewc_head.c`) for the board: both the forward pass

Table 1: Adaptation of the model under embedded constraints. The penalty of (1) is kept identical; what is simplified is its importance estimator and its optimization regime.

	PC (PyTorch)	Board (ewc_head.c)
Importance estimator	squared gradient, Eq. (2)	magnitude W^2 , Eq. (4)
Data required	one labelled batch per task	none
Accumulation	replaced at every consolidation	moving average, $\alpha = 0.99$
Penalty scope	weights and biases	weights only
Optimization	batches, momentum, several epochs	one sample, no momentum, single pass
λ	1000	400
Trigger	end of task	frame flag, or drift detection
Initial state of F	first computed at the end of task 1	zero, hence no pull before consolidation

and the Stochastic Gradient Descent (SGD) update run on the Cortex-M4 in FP32 (the F439ZI has no NPU). Weights are exported from the PyTorch checkpoint into a generated C header (`model_weights_ewc.h`) — never hand-edited — so that board and PC share *exactly* the same parameters.

3.3 Paired PC↔board protocol

To make the comparison honest, board and PC consume the same sequence: same seed (`seed = 42`), same sample order, same normalization (feature indices and arrays are produced by a single source). Two protocols are evaluated: *frozen* (inference only, weights fixed) and *online* (inference then continual update). *Parity* is the prediction-to-prediction agreement rate between the two platforms.

3.4 Quantization formulation

The mapping and its scale. Quantizing a tensor means replacing its real values by integers on a regular grid, plus a scale that maps that grid back to the floating-point world. For a weight on b bits in signed symmetric representation, with $q_{\max} = 2^{b-1} - 1$:

$$Q_{ji} = \text{clip}\left(\text{round}\left(\frac{W_{ji}}{s_j}\right), -q_{\max}, q_{\max}\right), \quad \widehat{W}_{ji} = s_j Q_{ji}. \quad (5)$$

The whole behaviour of a scheme lies in the choice of s_j . We compare three of them, which form a single family rather than separate methods:

$$s_j = \underbrace{\frac{1}{128}}_{\text{fixed}} \quad \text{or} \quad \underbrace{\frac{\max_{j,i} |W_{ji}|}{q_{\max}}}_{\text{per tensor}} \quad \text{or} \quad \underbrace{\frac{\max_i |W_{ji}|}{q_{\max}}}_{\text{per output channel}}. \quad (6)$$

The fixed scale is the one of the original embedded kernel. It does not depend on the weights: any coefficient with modulus above 127/128 is clamped onto the edge of the grid, and every coefficient smaller than the step collapses onto zero.

The integer layer. Activations receive the same treatment, with a scale s_a obtained from a calibration. A layer then reads as an integer accumulation followed by a single return to floating point:

$$\text{acc}_j = \sum_i Q_{ji} q_i^a, \quad y_j = \text{acc}_j \cdot s_j \cdot s_a + b_j. \quad (7)$$

Granularity is read off (7): the index j carried by s_j means that an output neuron whose weights are of small amplitude gets its own scale, instead of sharing the one set by the largest coefficient of the whole matrix. The calibration used is a per-layer absolute maximum, measured on a batch in floating-point precision ($s_a = \text{act_max}/q_{\max}^a$).

What the original scheme prescribes, and what we keep of it. The reference integer formulation [4, 6] aims at *fully* integer inference: the return to the next scale is done there by an integer multiplier and a shift, biases are carried in `int32` at the scale of the product, and a zero point offsets the activation grid. Our target has different constraints: the Cortex-M4F has a hardware floating-point unit, and the head has only three layers. We therefore keep the integer accumulation and the per-channel scales, but return to floating point between layers. Table 2 summarizes the divergences.

The format axis: the ablation ladder. The original embedded kernel accumulates four divergences from (5)–(7): a fixed scale, rounding replaced by truncation towards zero, an `int16` accumulator that silently wraps, and activations folded onto a Q7 grid. Rather than fixing them all at once, we walk a trajectory inside family (6), each step enabling one change only:

- `legacy_c`: fixed scale, truncation, `int16` accumulator (the original v1 kernel on the board);
- `fix_acc32`: accumulator widened to `int32`, everything else unchanged;
- `per_tensor_calib`: *calibrated* per-tensor scale and round-to-nearest;

Table 2: Reference integer scheme and embedded implementation. The divergences are dictated by the target (floating-point unit present, short network) and not by a convenience simplification.

	Reference [4]	v2 (<code>ewc_head_int8_v2.c</code>)	kernel
Scale return	integer multiplier and shift	floating-point dequantization on the hardware unit	
Biases	<code>int32</code> at the product scale	exact floats, never quantized	
Zero point	on activations and weights	symmetric weights with no zero point; symmetric activations, affine optional	
Granularity	per tensor, per channel as an extension	per output channel by default	
Calibration	histogram or percentile	per-layer absolute maximum on a batch	
Normalization fold- ing	required before quantization	not applicable, the network has none	
Accumulator	<code>int32</code>	<code>int32</code> , widened to <code>int64</code> for the Q15 variant	

- `per_channel_int8`: *per-output-channel* scale;
- `q15`: 16-bit grid, $q_{\max} = 32767$ (RAM $\div 2$ instead of $\div 4$).

The v2 firmware kernel implements the last two steps, with an optional Q15 variant (`-DEWC_INT8_Q15`); the v1 path is unchanged so that both can be compared on the same board. The ablation is *bit-exactly emulated on PC* (`src/utils/int8_c_emulation.py`) so as to isolate the contribution of each fix independently of board access. Scales are not recomputed on the board: they are produced on PC by the very functions the emulator uses, then written into a generated C header, which makes parity true *by construction* rather than observed after the fact.

Where quantization stops under the learning regime. A scope statement must be made before reading the "online" cells of Section 5, because it governs their interpretation. The integer kernel exposes a forward pass only: there is no gradient descent in the quantized domain. The board therefore keeps a *master* FP32 head, to which (3) and (4) apply, and whose integer representation is only a *view*, rebuilt through (5) after each step. Two consequences follow, and we state them rather than leave them implicit. First, the fourfold reduction of weight memory holds for inference alone: under the learning regime the board simultaneously carries the floating head, its integer view, the importance and the anchor, that is more than FP32 alone. Second, requantizing every matrix at each sample enters the latency budget, which Section 5.4 quantifies. In other words, this work establishes a quantization

compatible with incremental learning — inference is integer, and learning keeps working through it — not learning *carried out* in integer arithmetic, which remains an open problem.

What per-channel costs. A scale per output neuron is not free: it adds an array of floats parallel to each matrix, one four-byte value per output row. On the ported head, these scales bring the actual weight-memory gain below the factor of four advertised by the type change alone. That is the price of robustness: per-channel isolates the large-amplitude neurons, which would otherwise impose their scale on the whole layer.

3.5 The moment axis: when to quantize

At a fixed format, the same head can be quantized at three instants. *Before* training, through QAT: a *fake-quant* inserted in the loop makes the weights learn under the reduced-precision constraint. *After*, through PTQ: the trained FP32 model is converted once and for all. And *both*, that is a QAT training whose weights are then exported to the calibrated integer kernel. Only this last path is faithful to the actual deployment, since it is the embedded kernel that runs inference, not the quantization simulation of the training framework. The three moments are compared at identical model, dataset and seed.

3.6 The depth axis: bits, granularity, symmetry

Below INT8, three parameters describe a scheme: the number of *weight bits* (4, 2, ternary, binary), the *granularity* of the scale (per tensor or per output channel) and the *symmetry* (symmetric, or affine with a zero point). The first two are read directly off (5) and (6): going deeper means lowering $q_{\max}$, and changing granularity means changing the scope of the maximum that defines s_j .

At very low depths, the linear grid gives way to two dedicated schemes, which we take as they stand. In ternary [10], a per-channel threshold decides which weights survive, and the scale is the mean of the surviving weights alone:

$$\begin{aligned}\Delta_j &= 0.7 \cdot \overline{|W_{j\cdot}|}, & Q_{ji} &= \text{sign}(W_{ji}) \cdot \mathbf{1}[|W_{ji}| > \Delta_j], \\ \alpha_j &= \overline{\{|W_{ji}| : |W_{ji}| > \Delta_j\}}.\end{aligned}\tag{8}$$

In binary [12], no weight is set to zero any more and the scale is the mean modulus of the channel: $Q_{ji} = \text{sign}(W_{ji})$, $\alpha_j = \overline{|W_{j\cdot}|}$. Both schemes carry a per-channel scale *by construction*, so the granularity axis stops applying once they are reached.

Symmetry, for its part, concerns activations. After a positive activation function, the negative half of a symmetric grid is of no use, which costs all

the more as bits are scarce. The affine variant offsets the grid by a zero point:

$$q = \text{round}\left(\frac{a}{s_a}\right) + z, \quad \hat{a} = (q - z) s_a. \quad (9)$$

Weights, whose distribution is centred, stay symmetric with no zero point in every scheme we evaluate.

One point finally deserves to be stated, as it governs how the advertised gains must be read: as long as weights remain stored in an `int8_t` container, reducing the number of bits frees *no* byte. The memory gain exists only with effective *bit-packing*, which in turn costs unpacking at run time. We therefore systematically distinguish the *theoretical* RAM of a scheme from the *measured* RAM of the binary.

3.7 Measurement methodology

The quantities reported in Section 7 call for three distinct instrumentations.

Memory. The RAM of a binary is not the footprint of its weights. We report *total RAM* = `.data` + `.bss` + stack peak, the first two terms being read from the binary and the third measured at run time by stack painting, keeping the maximum between the inference phase and the update phase.

Latency. The Data Watchpoint and Trace (DWT) cycle counter times the kernel to the cycle. For the INT8 path it is instrumented in *three segments* — dequantization, integer Multiply-Accumulate (MAC), requantization — so as to answer not "is INT8 slower?" but "which item costs?". Cycles are reported raw, since conversion to microseconds truncates the information at this scale.

Energy. Current is measured with an X-NUCLEO-LPM01A probe through three independent methods: regression of current against inference rate, difference against a Wait For Interrupt (WFI) idle state, and a batch of inferences per frame isolating computation from the link cost. We report them *side by side* without ever averaging them: they do not measure the same boundary, and their disagreement is itself a result. Any unmeasured cell is reported as "to be measured" rather than estimated.

4 Experimental setup

Hardware target. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB Core Coupled Memory (CCM), no NPU). Latencies are measured with the DWT cycle counter; the static footprint is read from the binary and completed by the stack peak (§3). The latency budget is

Table 3: Provenance and nature of the reported measurements.

Campaign	Object	Nature
exp_S36	FP32 parity, INT8 legacy, latency, RAM	measured board
exp_S39_ablation	Ablation ladder, causal diagnosis	emulated PC
exp_S40_board_v2	Recovery, calibrated v2 kernel	measured board
exp_S46_board	Quantization moment, A/B	measured board
exp_S47_depth	Depth and granularity	emulated PC
exp_S48_summary	Bit-packing, actual <code>.bss</code>	measured board
exp_S49_ram	Total RAM by component	measured board
exp_S50_int8_latency	Latency by kernel segment	measured board
exp_S53_*	Energy, rate, frequency	measured board

100 ms per inference + update cycle (Gap 2). Samples travel over the Universal Asynchronous Receiver-Transmitter (UART) link frame by frame, each frame protected by a CRC: a zero error rate is the condition for the compared predictions to be the board’s own.

Datasets and CL scenarios. Two industrial datasets with contrasting scenarios: *Pronostia* (FEMTO bearings [11], *class-incremental* by normal/fault condition) and *Monitoring* (industrial equipment, *domain-incremental* by machine type). Two feature conditions are considered: **5feat** (embedded subset) and **all** (native features). On Monitoring, **5feat** and **all** coincide (4 native features).

Energy bench. Current is recorded with an X-NUCLEO-LPM01A probe, the board being powered by the probe itself. Campaigns are run at an imposed inference rate over a fixed window, with repetitions, so that the gap between two cells is attributable to the model and not to the host. A clock-frequency sweep completes the setup, the peripheral bus frequency being held constant so as not to shift the serial link rate.

Provenance of the measurements. Table 3 lists the campaigns and, for each, its nature. This distinction governs the reading of the whole article: a value *measured on-board* and a value *emulated on PC* do not differ in reliability — the emulator reproduces the C kernel’s arithmetic exactly — but they do not answer the same question.

Identical conditions and honesty rule. All comparisons share seed, order and normalization (§3). The board-v2 campaign covers the four per-channel cells (two datasets × two protocols); the **q15** format was not streamed on-board. Any unmeasured quantity is reported as "to be measured", with

Table 4: FP32 PC $\leftrightarrow$ board parity and performance (*measured on-board*, `exp_S36`). F1 is the board’s; it equals the PC’s at the decimals reported, frozen parity being 1.000. Monitoring has only four native features, so its `5feat` and `all` conditions coincide and yield a single row.

Dataset	Cond.	Board F1	Frozen parity	Online parity
Pronostia	5feat	0.9164	1.000	0.9754
Pronostia	all	0.9180	1.000	0.9626
Monitoring	5feat	0.9194	1.000	0.9887

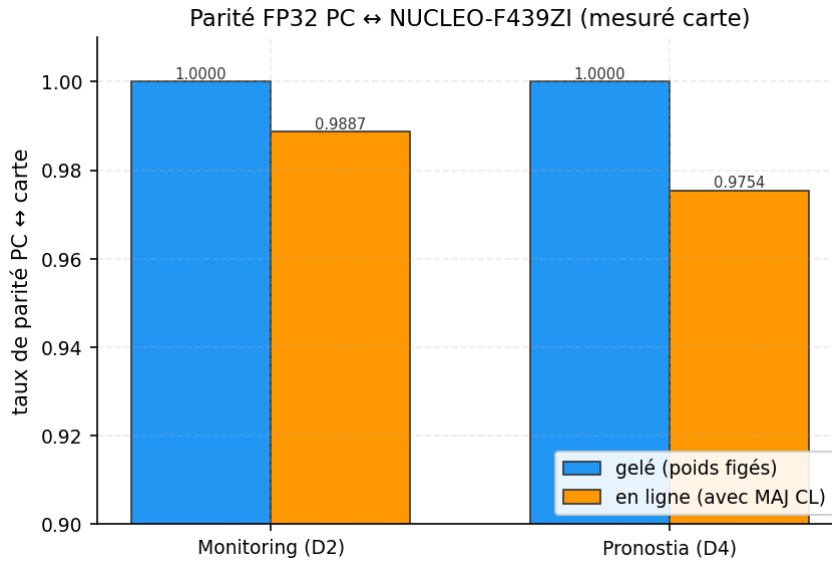


Figure 1: FP32 PC $\leftrightarrow$ board parity: frozen (1.000) vs online. *Measured on-board* (`exp_S36_parity_*.json`).

no invented value and no substituted estimate; the full registry of these cells, with the reason for each, is given in Appendix C. Acronyms used in the article are expanded at first use and collected in the list of acronyms, page 31.

5 Results

5.1 FP32 PC $\leftrightarrow$ board parity (measured on-board)

In FP32, the prediction-by-prediction parity is *exact* in the frozen protocol (1.000) on both datasets, and near-exact online (0.9754 Pronostia, 0.9887 Monitoring) — the online gaps stem from the board’s `float32` versus the PC’s `float64` at decision boundaries. Table 4 summarizes; Figure 1 illustrates.

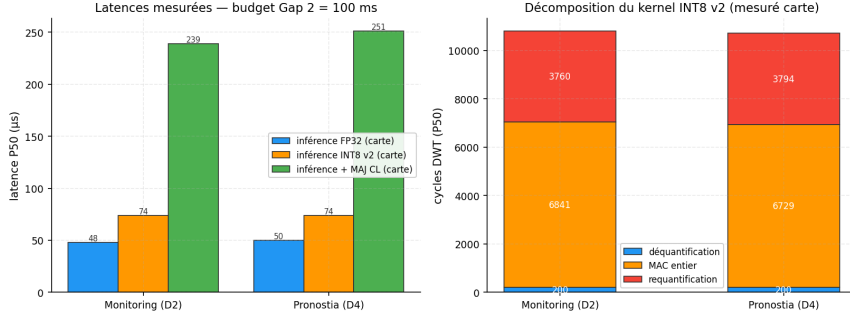


Figure 2: Board latencies (inference vs inference + update) against the Gap 2 budget (100 ms). *Measured on-board* (`exp_S36_summary.json`).

Table 5: INT8 *legacy* vs FP32 on-board (*measured*, `exp_S36`). RAM $\div 4$ but collapsed F1.

Dataset	F1 FP32	F1 INT8 legacy	INT8/FP32 agr.	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

Latency and RAM (Gap 2/Gap 3). All board latencies stay three orders of magnitude below budget: inference 50 μ s (Pronostia) / 48 μ s (Monitoring); inference + online update 251 μ s / 239 μ s, versus ≈ 33 –35 μ s for the reference PC inference. The `.bss` footprint ranges over 100–145 kB, below the 256 kB available. Figure 2 places these values against the 100 ms line.

5.2 Collapse of naive PTQ (measured on-board)

The *legacy* INT8 quantization of the binary EWC head collapses on the board: F1 drops from ≈ 0.92 (FP32) to 0.138 (Pronostia) and 0.1337 (Monitoring), for an INT8 $\leftrightarrow$ FP32 agreement of only ≈ 0.60 –0.74. Weight RAM is indeed divided by 4, but the metric is destroyed (Table 5).

5.3 Diagnosing the collapse (bit-exact PC emulation)

The measured collapse does not tell *what* caused it. A *bit-exact PC-emulated* ablation — an exact reference for the kernel’s arithmetic — isolates the contribution of each fix (Table 6, Figure 3). The dominant factor is the *calibrated per-tensor scale* on an `int32` accumulator: `per_tensor_calib` restores +0.88 F1 (Pronostia 0.9462, Monitoring 0.9201). The `q15` format exactly matches FP32.

The cause is thus identified, and the emulator *predicts* recovery. Whether the corrected kernel actually delivers it once flashed is the subject of the next subsection, where the question moves from emulation to measurement.

Table 6: INT8 ablation ladder (F1, *bit-exact PC emulation*, exp_S39_ablation).

Scheme	Pronostia F1	Monitoring F1
FP32 (ref.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

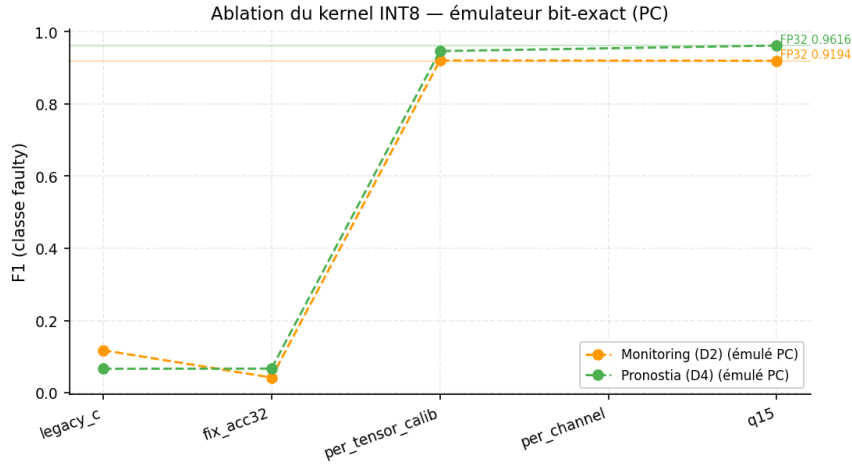


Figure 3: Ablation ladder `legacy_c`→`per_tensor_calib`→`per_channel`→`q15`. *Bit-exact PC emulation* (exp_S39_ablation).

5.4 Recovery measured on-board (calibrated v2 kernel)

Recovery is no longer an emulator prediction: the calibrated-scale v2 kernel was flashed onto the NUCLEO-F439ZI and streamed over the four per-channel cells (two datasets $\times$ two protocols). F1 returns to the FP32 level — 0.9173 against 0.9194 in FP32 on Monitoring, 0.8995 against 0.9164 on Pronostia — while weight RAM stays divided by 4. Parity against the bit-exact emulator is *exact* in the frozen protocol (1.000), which validates a lossless port, and no CRC error was observed (Table 7).

The latency overhead is far heavier under learning than under inference. The latency column of Table 7 carries the sharpest figure of this campaign, and it does not concern the frozen mode. Under inference alone, INT8 costs 65 and 68 μ s against 48 and 50 in FP32, the roughly one-half overhead that Section 7 breaks down item by item. Under continual learning the gap changes order: 577 and 606 μ s against 239 and 251 in

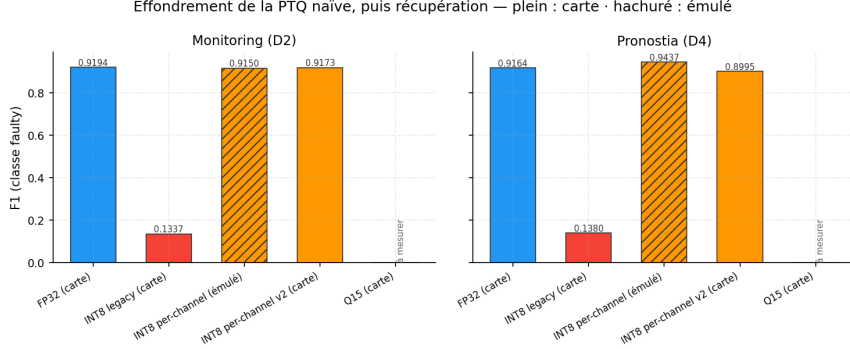


Figure 4: INT8 vs FP32: *legacy* collapse then recovery. Collapse and recovery are both *measured on-board* (Sprints 36 and 40); the *PC-emulated* per-channel column (Sprint 39) is recalled with hatching. The **q15** format on-board remains "to be measured".

Table 7: INT8 recovery *measured on-board* (per-channel v2 kernel, **exp_S40_board_v2**). Parity = agreement with the bit-exact emulator; agreement = INT8 against FP32 on the same samples.

Dataset	Protocol	F1	Parity	Agreement	Latency (μ s)	.bss (B)
Monitoring	frozen	0.9173	1.000	0.9996	65	101236
Monitoring	online	0.9016	0.9885	0.8337	577	101236
Pronostia	frozen	0.8995	1.000	0.9951	68	106152
Pronostia	online	0.9212	0.9736	0.8100	606	106152

FP32, a factor of 2.4. The cause is the one set out in §3: every gradient step applies to the master FP32 head, and the integer view must be rebuilt in full behind it. The Gap 2 budget still holds with two orders of magnitude to spare, but the reading is unambiguous — **it is precisely in the regime that continual learning imposes that quantization costs the most**. Requantizing only the rows actually modified by the step is the natural way to bring that factor back near one; it remains to be implemented.

Two caveats deserve to be stated. First, the INT8 $\leftrightarrow$ FP32 agreement drops online (0.8337 and 0.8100 against > 0.99 frozen): the continual update makes the two weight trajectories diverge, without the final metric suffering from it. Second, the **q15** format and the A/B rerun of the **int8_legacy** scheme under the v2 kernel were not streamed: those cells remain "to be measured", in line with the "no invented number" rule.

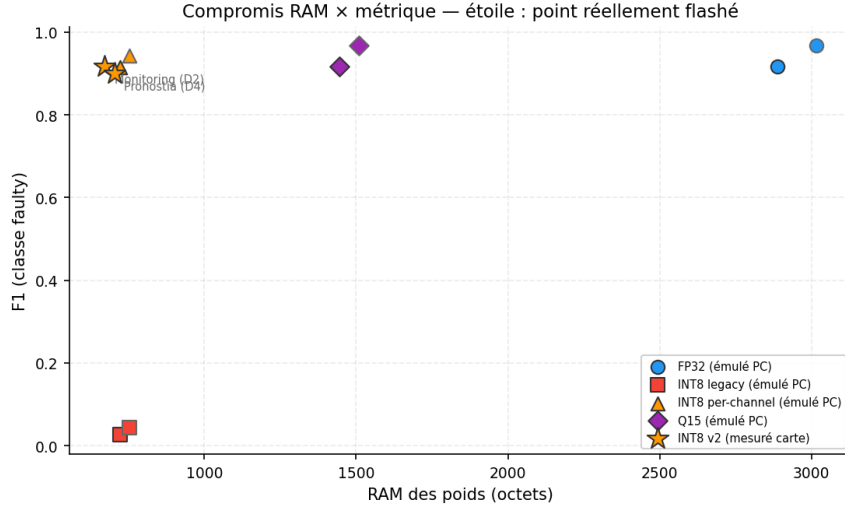


Figure 5: RAM $\times$ F1 $\times$ latency trade-off front. Mixed *measured/emulated* sources annotated per point.

5.5 RAM $\times$ F1 $\times$ latency trade-off

Figure 5 synthesizes the trade-off: the calibrated kernel brings the metric back to FP32 level while keeping the RAM gain (INT8 $\div$ 4, Q15 $\div$ 2), with no prohibitive latency cost. The star marks the point actually flashed (per-channel v2 kernel); the other points are emulated. The expected latency gain, however, does not materialize — we return to this in Section 8.

6 Two further quantization axes: moment and depth

The previous sections vary the *calibration* of the kernel at a fixed format. Two further degrees of freedom drive the outcome: *when* one quantizes, and *how many bits* one keeps. The three placements compared here, and the parameters of a sub-INT8 scheme, are defined in §3.

6.1 Moment: before, after, or both

Three placements are possible for the same EWC head. *Before* training (QAT, fake-quant in the loop); *after* (PTQ on an already trained FP32 model); and *both*, that is a QAT run whose weights are then exported to the calibrated integer kernel — only this last path is faithful to the actual deployment.

Under emulation, all three moments preserve Area Under the ROC Curve (AUROC) on Monitoring (0.974477 before, 0.974649 after, 0.974663 both, against 0.97464 in FP32): at this model scale no moment is disqualifying.

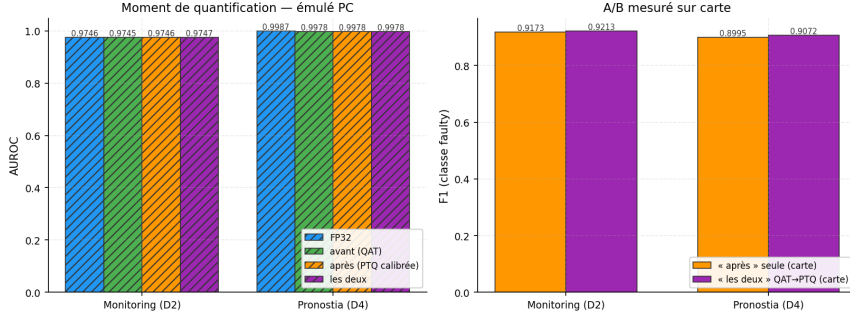


Figure 6: Quantization moment. Left, the three moments *PC-emulated* (hatched); right, the *on-board measured* A/B between calibrated PTQ alone and the QAT→PTQ path.

The question is settled *on-board*. The "both" path was flashed and streamed: it yields 0.9213 F1 on Monitoring and 0.9072 on Pronostia, against respectively 0.9173 and 0.8995 for the calibrated PTQ alone (Section 5.4).

The gap, 0.004 and 0.0077, is real but small. The honest reading is therefore that **kernel calibration recovers what matters, and that QAT adds no decisive gain** on this head: when the training pipeline cannot be modified, a properly calibrated PTQ suffices. This contradicts the common reading whereby a PTQ collapse would necessarily call for retraining.

6.2 Depth: how far below 8 bits

Below 8 bits the question becomes: at which depth does the metric break down, and does the granularity of the scale factor move that limit? The emulated sweep answers both.

The limit depends on the dataset; the full grid is in Appendix A (Table 9). On Monitoring, degradation stays contained down to binary ($\Delta\text{AUROC} = -0.0117$). On Pronostia, binary breaks down (-0.0275) but ternary holds (-0.0153): read in AUROC, the usable frontier would therefore be *ternary*, binary being defensible only on the easier dataset.

The frontier depends on the selection metric. That reading does not survive confrontation with the *on-board measured* F1 of the very same schemes, which Table 8 places next to AUROC. On Pronostia, ternary keeps an AUROC of 0.99907 but its F1 falls to 0.379; binary keeps an AUROC of 0.96616 for a *null* F1, that is, a model that never predicts the fault class again. The ranking of scores stays good where the decision threshold no longer does: a ranking metric is insensitive to class imbalance, an F1 is not. On both datasets, the only sub-INT8 scheme that genuinely preserves the operational metric is **INT4** (0.921 and 0.9887).

We report both readings rather than picking one, because their divergence is itself the result: **it is not depth alone that sets the frontier, it is the depth-selection-metric pair**. Work reporting AUROC only would conclude ternary and deploy, on Pronostia, a detector that detects nothing.

Table 8: Sub-INT8 schemes *measured on-board*: AUROC and **faulty**-class F1 on the same cells. AUROC stays high where F1 breaks down, depth not moving the two metrics together.

Depth	Monitoring		Pronostia	
	AUROC	F1	AUROC	F1
int4	0.98787	0.9210	0.99995	0.9887
ternary	0.98409	0.9042	0.99907	0.3790
binary	0.96784	0.6878	0.96616	0.0000

Granularity, in turn, pushes the breakdown back. At 2 bits on Pronostia, moving from a per-tensor to a per-channel scale takes Δ AUROC from -0.0464 to -0.0086 . A negative result deserves reporting too: the affine *zero-point* buys nothing — at 2 bits on Monitoring it even degrades markedly (-0.0619 against -0.0069 symmetric), and Table 10 shows the finding holds at all three depths tested.

The RAM gain only exists packed. The crucial point is that the RAM advertised by these schemes is *theoretical* as long as weights stay in an **int8** container. Measured on-board, the **.bss** of a 4-, 2- or 1-bit *unpacked* model is strictly identical to the INT8 one: the gain only appears with genuine bit-packing, where it grows as bits drop (336, 504 then 572 bytes saved on Monitoring). Unpacking is paid in latency, about 55 μ s (from 67 μ s unpacked to 123 μ s packed on Monitoring) — without threatening the Gap 2 budget. Parity against the emulator stays 1.000 across the twelve measured cells: the scheme is ported losslessly.

7 System budget: memory, latency, energy, compute

A quantization gain cannot be judged on weights alone. This is where the threefold argument recalled in the introduction — less memory, less time, less energy — is settled: this section gathers the four budgets of one and the same deployed EWC head, all measured on the same board, and takes each promise separately. The three corresponding instrumentations (RAM components, cycle-level kernel segmentation, energy estimators) are described in §3.

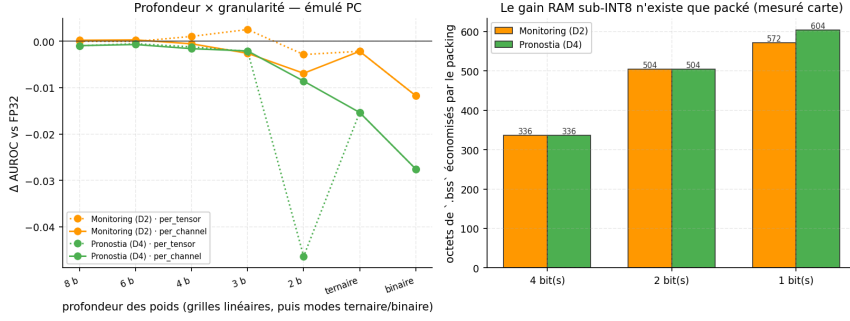


Figure 7: Sub-INT8 depth. Left, ΔAUROC by depth and granularity (*PC-emulated*) — the on-board measured F1 reading, which moves the frontier, is in Table 8; right, the `.bss` bytes actually saved by bit-packing (*measured on-board*).

7.1 Weight RAM is not system RAM

INT8 does divide *weight* RAM by 4 (2688 to 672 bytes on Monitoring, 2816 to 704 on Pronostia). This count covers the weight matrices alone, the ones quantization touches; biases stay floating and exact (§3), so the head’s parameter count, 722 and 754, is slightly higher. But the RAM actually consumed by the system is `.data` + `.bss` + stack peak, and that sum barely moves: it goes from 105300 to 105324 bytes on Monitoring, an INT8/FP32 ratio of 1.0002. The per-component breakdown is in Appendix B (Table 11). Both configurations occupy 40.17 % and 40.18 % of the 256 kB budget (42.03 % and 42.04 % on Pronostia).

The explanation is simple and worth stating plainly: at this model scale, weights are not the dominant item — streaming buffers, the stack and the rest of the firmware are. Advertising a “÷4 RAM” without saying *which* RAM would therefore be misleading. The ÷4 is real and useful, but it applies to a minority item of *this* system; it would become decisive on a model where weights dominate.

7.2 Where INT8 latency goes

Cycle-level instrumentation of the INT8 kernel quantifies the announced paradox. On Monitoring, the integer MAC costs 6841 cycles, requantization 3760 and dequantization 200 (respectively 6729, 3794.5 and 200 on Pronostia). The total reaches 74 μs against 48 μs in FP32 on Monitoring and 50 μs on Pronostia.

The decisive item is therefore not multiplication: it is *requantization*, a third of the budget, and it simply does not exist in the FP32 path. The integer MAC, for its part, brings no gain against the Cortex-M4 hardware FPU. The INT8 overhead is not an implementation anomaly but the arith-

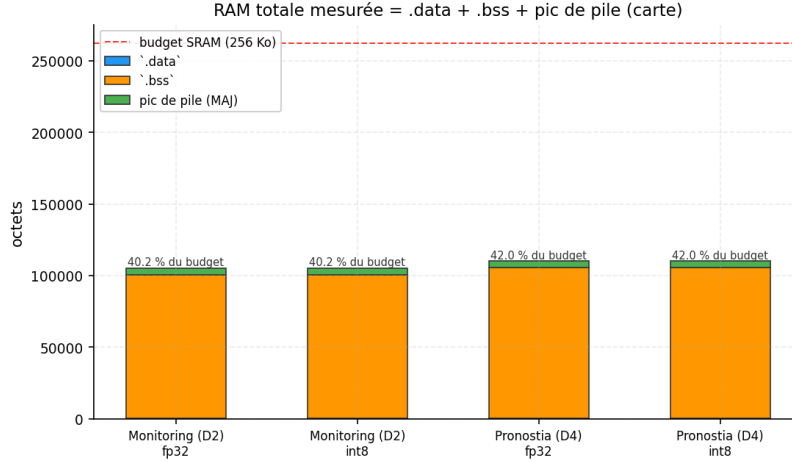


Figure 8: Total RAM *measured on-board*: `.data` + `.bss` + stack peak, against the SRAM budget. The INT8 total equals the FP32 total, even though weights are divided by 4.

metric consequence of an integer format executed on a target that natively computes in floating point.

7.3 Energy: three estimators, never merged

Energy was measured with the X-NUCLEO-LPM01A probe by three independent methods, which we report *side by side* without ever averaging them — their disagreement is itself a result. Regressing current against rate gives $67.65 \mu\text{J}$ per inference in FP32 and $67.47 \mu\text{J}$ in INT8; the delta against a WFI idle gives $146.77 \mu\text{J}$ and $145.03 \mu\text{J}$; the batch of inferences per frame, which isolates compute from the UART frame cost, gives $5.08 \mu\text{J}$. The Table 12 lines them up side by side. The order of magnitude shared by the first two, and the factor-of-ten gap with the third, delimit exactly what each one measures: the first two include the communication cost, the last does not.

On the question at hand the three agree: **INT8 does not reduce consumption**. The measured gap is $0.18 \mu\text{J}$ for a combined uncertainty of $2.34 \mu\text{J}$, far below the significance threshold. The latency paradox is thus compounded by an energy paradox, and the case for INT8 reduces to memory.

The frequency sweep, in contrast, opens a real lever. Energy per inference grows monotonically with the clock — $170.05 \mu\text{J}$ at 45 MHz, $192.64 \mu\text{J}$ at 90 MHz, $214.61 \mu\text{J}$ at 180 MHz — while latency stays far below the Gap 2 budget, up to $198 \mu\text{s}$ at the slowest point. In other words, *latency headroom is convertible into battery life*: at the most frugal measured regime, a 2000 mAh cell lasts 72.95 h at one inference per second. Two cells remain

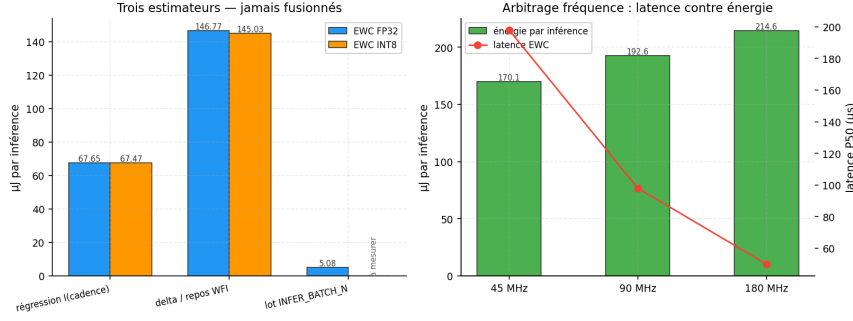


Figure 9: Energy *measured on-board*. Left, the three estimators side by side; right, the frequency trade-off, where latency headroom is exchanged for energy.

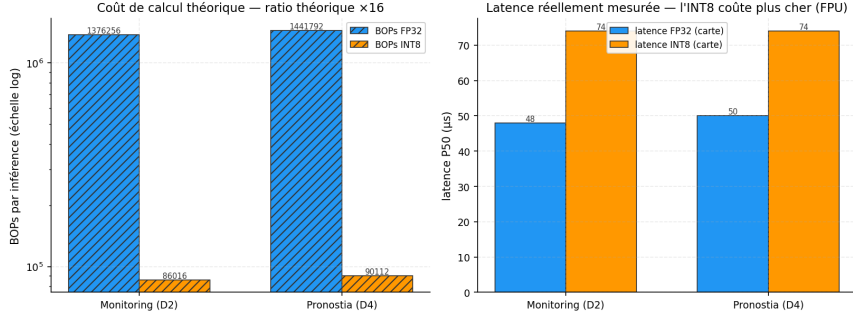


Figure 10: *Theoretical* compute cost (left, hatched) against *on-board measured* latency (right). The $16\times$ BOP gain translates into no time gain at all.

"to be measured": energy per continual update, whose regressions are not publishable as they stand, and the per-phase profile.

7.4 Compute cost

Analytically, the head has 722 parameters on Monitoring (754 on Pronostia), for 672 MACs and 1344 Floating-Point Operations (FLOPs) per inference (704 and 1408). Counting *binary operations* gives the expected theoretical ratio: 1376256 BOPs in FP32 against 86016 in INT8, a factor of 16 — that is, $(32/8)^2$.

This factor of 16 is precisely what the board does *not* deliver. Contrasting the two columns of Figure 10 is the most direct way to see why an analytical cost metric cannot replace a measurement: a Bit Operation (BOP) counts operand bits, not cycles of an existing floating-point unit.

8 Discussion

The INT8 latency paradox, quantified. INT8 quantization divides the weight memory footprint by 4 but does *not* speed up inference on the Cortex-M4 — it slows it down. Cycle-level instrumentation of the kernel (Section 7) says where: across both datasets, the integer MAC costs on average 6785 cycles and requantization 3777.25, while dequantization weighs only 200. The INT8 total settles at 74 μ s against 48–50 μ s in FP32. Two causes therefore compound: the integer MAC has nothing to gain against a hardware FPU already efficient in single precision, and requantization — an item that does not exist in the floating-point path — consumes a third of the budget on its own. The direct lead remains a port to integer Single Instruction, Multiple Data (SIMD) kernels, such as those of Common Microcontroller Software Interface Standard — Neural Network (CMSIS-NN) [8], which would attack the first item; the second rather calls for a power-of-two scale chain. This balance concerns inference. Under continual learning the overhead is heavier still, by a factor of 2.4 (Section 5.4), since rebuilding the integer view follows every gradient step. That is the regime on-device learning claims, and therefore where the paradox weighs most.

An energy paradox compounds the latency paradox. One could hope that INT8, failing to be faster, would at least be leaner. Measurement says otherwise: 67.47 μ J per inference in INT8 against 67.65 μ J in FP32, a gap of 0.18 μ J for a combined uncertainty of 2.34 μ J. There is thus no measurable effect. The practical consequence is clear: on this target, **INT8 is justified by memory, neither by speed nor by energy**. The energy lever lies elsewhere — in clock frequency, where the ample latency headroom is exchanged for battery life.

"Measured" vs "emulated" honesty. We rigorously separate two kinds of results. FP32 parities, the legacy INT8 collapse, recovery through the v2 kernel, latencies, RAM and energy are *measured on real hardware* (Sprints 36, 40, 46 to 50 and 53). The causal diagnosis, the ablation ladder and the depth sweep are *bit-exactly emulated on PC* (Sprints 39 and 47): the emulator reproduces the C kernel arithmetic exactly, which makes it a reliable reference, but it is not a board measurement. Since the v2 campaign, recovery no longer belongs to the second category: it is established on the four per-channel cells, with a frozen parity of 1.000 against the emulator. What remains unmeasured — the q15 format on-board, energy per continual update, the per-phase profile — is reported as "to be measured" rather than estimated.

Limitation: forgetting is not stressed on these two datasets. Our continual scenarios there are short (three tasks) and average forgetting stays

below 0.01: these datasets do not put EWC regularization to the test. Catastrophic forgetting was measured elsewhere in the project, on CWRU, where it reaches 0.8475 for EWC against 0.8578 under naive learning. As that dataset does not overlap ours, we refrain from mixing the two findings: this article’s contribution bears on *porting* and *quantization*, not on the effectiveness of regularization against forgetting.

Scope. The central result — a naive PTQ can destroy a binary decision head, and a per-tensor/per-channel calibration (or a Q15 format) restores it — extends beyond the EWC case: it concerns any lightweight head quantized downstream of a feature extractor, a frequent regime in TinyML.

9 Conclusion and future work

We ported a predictive-maintenance EWC classifier onto a Cortex-M4 micro-controller and established, under a paired protocol, an exact FP32 PC $\leftrightarrow$ board parity in the frozen protocol, with an inference + online learning latency $\ll 100$ ms and a footprint below 256 kB (Gaps 2 and 3). On that same board we *measured* the collapse of naive INT8 quantization of the binary head, *isolated its cause* through a bit-exact emulated ablation — an uncalibrated per-tensor scale on too narrow an accumulator — then *measured its recovery* with a calibrated kernel, with F1 back to 0.9173 and 0.8995 and a frozen parity of 1.000 against the emulator. The diagnosis remains emulated; the recovery no longer is.

Two complementary axes sharpen this result. On the quantization *moment*, kernel calibration recovers what matters and QAT training adds no decisive gain: when the training pipeline cannot be modified, a properly calibrated PTQ suffices. On *depth*, the boundary is not set by bits alone but by the depth–metric pair: ternary holds in AUROC and breaks down in on-board measured F1, where the boundary returns to INT4. Per-channel granularity pushes back the breakdown, and the memory gain advertised below 8 bits exists only with genuine bit-packing — which costs about 55 μ s of unpacking, without threatening the budget.

What INT8 delivers, and what it does not. The measured system budget decouples the three usual promises. Weight RAM is indeed divided by 4, but total system RAM is unchanged (ratio 1.0002), because at this model scale the weights are not the dominant item; and that $\div 4$ holds for inference, the floating head staying in memory as soon as the board learns. Latency *increases* by about half under inference and by a factor of 2.4 under continual learning, requantization — an item absent from the floating-point path — consuming one third of the inference budget on its own. Energy, finally, does not move: the measured gap is an order of magnitude below its

uncertainty. On this target INT8 is therefore justified by memory alone, and the energy lever lies elsewhere — in the clock frequency, where the latency margin we amply hold can be traded for battery life.

Future work. The latency paradox points toward integer SIMD kernels (CMSIS-NN) for the MAC item, and toward a power-of-two scale chain for the requantization item; this is the most direct route for the memory gain to stop being paid in time. The learning regime calls for two routes of its own: requantizing only the rows modified by the gradient step, which attacks the factor of 2.4 directly, and running gradient descent in the integer domain, the only way for the memory gain to bear on learning itself rather than on inference alone. The "total RAM does not move" finding further calls for a replication on a model where the weights *do* dominate the footprint, a regime in which the $\div 4$ would become decisive again. Finally, three cells remain explicitly "to be measured": the `q15` format on-board, the energy per online update, and the per-phase energy profile. We leave them empty rather than estimate them — the same rule that, throughout this article, separates what is measured on-board from what is emulated on PC.

Code and data availability

The C firmware, the quantization kernels, the bit-exact emulator, the campaign drivers and the figure scripts belong to the CL-Embedded project repository. Every value reported here comes from a versioned result file, gathered without recomputation into a single aggregate from which figures and tables are derived: no value is hand-entered in the text or in the figures. The datasets are public and cited in §4.

Acknowledgements

This work was carried out at ISAE-SUPAERO (DISC), in collaboration with ENAC (LII) and Edge Spectrum. The author thanks his supervisors for their reviews and their guidance on embedded quantization and on the conduct of the measurement campaigns.

References

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.

- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance: Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [5] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
- [6] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. arXiv:1806.08342, 2018.
- [7] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner: Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
- [8] Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs. arXiv:1801.06601, 2018.
- [9] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
- [10] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. arXiv:1605.04711, 2016.
- [11] Patrick Nectoux, Rafael Gouriveau, Kamal Medjaher, Emmanuel Ramasso, Bechir Chebel-Morello, Nouredine Zerhouni, and Christophe Varnier. PRONOSTIA: An experimental platform for bearings accelerated degradation tests. In *IEEE International Conference on Prognostics and Health Management (PHM)*, 2012.

- [12] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *European Conference on Computer Vision (ECCV)*, 2016.
- [13] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 11(4):789–802, 2021.
- [14] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL: TinyML with online-learning on microcontrollers. arXiv:2103.08295, 2021.
- [15] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress: A scalable framework for continual learning. In *International Conference on Machine Learning (ICML)*, 2018.

A Depth and symmetry sweep

Section 6 quotes only the decision points of the sub-INT8 sweep. Tables 9 and 10 give the full grid in Δ AUROC, from which that metric’s ternary boundary and the uselessness of the affine zero point follow. The on-board measured F1 of the same schemes, which moves that boundary to INT4, is given in Table 8.

Table 9: Full depth sweep: Δ AUROC against the FP32 reference, by depth and granularity (*bit-exact PC emulation*). A negative Δ is a degradation.

Depth	Monitoring		Pronostia	
	Per tensor	Per channel	Per tensor	Per channel
int8	0.0000	0.0002	-0.0009	-0.0009
int6	-0.0000	0.0003	-0.0005	-0.0007
int4	0.0011	-0.0005	-0.0012	-0.0016
int3	0.0026	-0.0026	-0.0021	-0.0021
int2	-0.0028	-0.0069	-0.0464	-0.0086
ternaire	-0.0021	-0.0021	-0.0153	-0.0153
binaire	-0.0117	-0.0117	-0.0275	-0.0275

Table 10: Quantization symmetry: Δ AUROC, symmetric against affine with a zero point (*bit-exact PC emulation*).

Scheme	Monitoring		Pronostia	
	Symmetric	Affine	Symmetric	Affine
int4	-0.0005	-0.0554	-0.0016	-0.0019
int3	-0.0026	-0.0591	-0.0021	-0.0028
int2	-0.0069	-0.0619	-0.0086	-0.0125

B System budget in detail

Table 11 breaks down the RAM that Section 7 only aggregates into a total: it shows directly that `.bss` is *identical* between FP32 and INT8, the difference in total coming from the stack peak alone. Table 12 lines up the three energy estimators.

Table 11: RAM components, in bytes (*measured on-board*). The total is the sum of the first three rows, the stack peak being the maximum of the two phases.

Component	Monitoring		Pronostia	
	FP32	INT8	FP32	INT8
<code>.data</code>	460	460	460	460
<code>.bss</code>	100152	100152	105036	105036
Stack peak (inference)	4416	4328	4424	4336
Stack peak (update)	4688	4712	4696	4720
Total	105300	105324	110192	110216
Ratio	1.0002		1.0002	

Table 12: The three energy estimators, side by side and never merged (*measured on-board*). In μ J per inference.

Estimator	FP32	INT8
Rate regression	67.65	67.47
Difference against WFI idle	146.77	145.03
Batch of inferences per frame	5.08	—

C Registry of missing measurements

Table 13 lists the cells the article explicitly leaves "to be measured", with the reason for each. Publishing this registry is the counterpart of the "no invented number" rule: it makes the absence verifiable instead of leaving it to be guessed.

Acronyms

Area Under the ROC Curve (AUROC) Area under the ROC curve: the probability that an anomalous sample receives a higher score than a normal one. Independent of the decision threshold. 16

Bit Operation (BOP) Cost expressed at the bit level, proportional to the product of operand widths; it predicts a factor of 16 between FP32 and INT8 that the board does not deliver, illustrating the limits of an analytical metric. 21

Common Microcontroller Software Interface Standard — Neural Network (CMSIS-NN) Library of neural-network kernels optimized for Cortex-M cores, exploiting vector instructions; the direct route to lifting the latency paradox observed here. 22

Continual Learning (CL) Learning a sequence of tasks without access to past data, where the model must absorb the new without erasing the old. 2

Core Coupled Memory (CCM) Fast memory attached directly to the processor core, not reachable by some peripherals; on the target board it complements the main static memory. 10

Cyclic Redundancy Check (CRC) Checksum computed on every transmitted frame; a zero error rate guarantees that the compared predictions are those produced by the board and not the product of a corrupted transmission. 2

Data Watchpoint and Trace (DWT) Debug unit of the Cortex-M core whose cycle counter times a code section to the cycle, then converts it to microseconds from the clock frequency. Every latency in this article comes from it. 10

Elastic Weight Consolidation (EWC) Anti-forgetting method adding to the loss a quadratic penalty that restrains changes to the weights deemed important for past tasks, importance being estimated by the

Table 13: Registry of unmeasured cells. They carry "to be measured" in the text and are never replaced by an estimate.

Cell	Reason
energy.estimators.delta_wfi.cellspagne.fp32	différents énergysuleperupdate la MAJ CL exige la même mesure avec...
energy.estimators.delta_wfi.cellspagne.int8	différents énergysuleperupdate la MAJ CL exige la même mesure avec...
energy.estimators.policy_update.cellspagne.fp32	différents énergysuleperupdate sion de always, frozen n'est pas...
energy.estimators.policy_update.cellspagne.int8	différents énergysuleperupdate sion de always, frozen n'est pas...
monitoring.ram.fp32_pc_bss	valeur absente de experi- ments/exp_S49_ram/summary.json
monitoring.ram.fp32_pc_data	valeur absente de experi- ments/exp_S49_ram/summary.json
monitoring.ram.fp32_pc_stack	valeur absente de experi- ments/exp_S49_ram/summary.json
monitoring.ram.int8_pc_bss	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_data	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_stack	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_stack_peak	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_stack_peak_update	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_total	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.recovery_board.int8_legacy	cellulaire frozen S4002 faulty streamée (carte requise)
monitoring.recovery_board.int8_legacy_online	cellulaire frozen S4002 faulty streamée (carte requise)
monitoring.recovery_board.q15_chillzone_f1	cellulaire frozen S4002 non streamée (carte requise)
monitoring.recovery_board.q15_chillzone_f1_grille5	cellulaire frozen S4002 non streamée (carte requise)
pronostia.ram.fp32_pc_bss	valeur absente de experi- ments/exp_S49_ram/summary.json
pronostia.ram.fp32_pc_data	valeur absente de experi- ments/exp_S49_ram/summary.json
pronostia.ram.fp32_pc_stack	valeur absente de experi- ments/exp_S49_ram/summary.json
pronostia.ram.int8_pc_bss	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...

Fisher information. It stores no past data, which makes it compatible with a microcontroller. 2

Floating-Point Operation (FLOP) Elementary floating-point operation; one multiply-accumulate counts as two. 21

Floating-Point Unit (FPU) Hardware floating-point unit; on the target board it makes FP32 operations as fast as integer ones, which cancels the speed benefit expected from quantization. 4

Hyperdimensional Computing (HDC) Non-neural paradigm encoding signals into very high-dimensional vectors; learning reduces to elementary operations (addition, majority vote), without backpropagation. 3

Microcontroller Unit (MCU) Processor integrating core, memory and peripherals on a single chip, intended for self-contained embedded systems. 2

Multiply-Accumulate (MAC) Elementary operation multiplying two values and adding the product to an accumulator; the counting unit for the cost of a dense layer. 10

Neural Processing Unit (NPU) Circuit specialized in neural-network operations, able to run integer multiply-accumulates massively in parallel. Its absence on the target board explains why INT8 brings no speed gain there. 2

Post-Training Quantization (PTQ) Quantization applied to an already trained model: simple to implement, but prone to losses when the scale is set without calibration or when tensor dynamics are wide. This is the regime whose collapse and recovery this article measures. 2

Quantization-Aware Training (QAT) Quantization simulated at every training step through *fake-quant*, so that the model learns directly under reduced-precision constraints; costlier to train, it brings no decisive gain here over a properly calibrated PTQ. 2

Random Access Memory (RAM) Volatile memory holding the program's variables and buffers; the most critical resource of the target. 2

Single Instruction, Multiple Data (SIMD) Instruction set applying one operation to several data items in a single cycle; this is the lever the integer path of this work lacks to turn a format gain into a time gain. 22

Static Random Access Memory (SRAM) Static volatile memory integrated in the microcontroller, fast but expensive in silicon area, hence available in very small amounts. It sets the memory budget of this work. 2

Stochastic Gradient Descent (SGD) Optimization algorithm updating the weights along the direction opposite to the gradient, estimated on a small batch of examples; this is the update run on-board in the online protocol. 6

Tiny Machine Learning (TinyML) Machine learning on microcontrollers, characterized by three cumulative constraints: no external memory, no operating system, and less than 256 kB of volatile memory. 3

Tiny Online Learning (TinyOL) Approach where a pretrained network is frozen in Flash and only a lightweight trainable layer is updated in RAM, sample by sample. 3

Universal Asynchronous Receiver-Transmitter (UART) Asynchronous serial link between board and host; it carries the samples frame by frame and reports the predictions. 11

Wait For Interrupt (WFI) Instruction putting the core to sleep until the next interrupt; using it while waiting on the serial link lowers the idle current and makes the difference-based energy measurement usable. 10